PERSONAL SKILLS · ~/.CLAUDE/SKILLS

My Skills

a field guide — what each one does, and when it fires.

Seven personal skills, mirrored to `~/skills` and version-controlled. Ordered as a project flows — plan, build, run, design, then share, document, and illustrate. Each is invoked automatically when its trigger appears, or on request by name.

// 01
manifest
Plan before code; a living task doc.

// 02
stacked
Decompose into reviewable layers.

// 03
cmux
Drive terminals & browser by CLI.

// 04
kagaz
Editorial-technical design + PDF system.

// 05
dak
Local artifact → shareable URL.

// 06
how-to
Styled walkthrough guide PDFs.

// 07
ian-illustrations
Xiaohei hand-drawn article art.

manifest

write the plan before the code, then keep it honest.

WHAT IT DOES

A living markdown doc per non-trivial task at `tasks/<slug>/manifest.md` — capturing the **problem, decisions, exploration findings, plan, open questions, test & rollout plans, and a final review**. Updated continuously as the work progresses, so the deliverable is the code *and* the doc that explains it.

HOW TO USE

Fires at the **start** of a feature, migration, refactor, or any 3+ step task — "build X", "migrate X", "let's plan". At init it creates **only** `manifest.md`; `data/`, `artifacts/`, `runs/` are added *lazily* when first needed. `tasks/` is gitignored — local-only. *Not for* typos, one-line fixes, or questions.

stacked

small, dependent, independently reviewable.

WHAT IT DOES

Breaks a large, complex, or risky change into a **stack of small units** — each with one responsibility, each building on the layer below — so every PR answers a single reviewable question instead of arriving as one unreadable diff.

HOW TO USE

Fires on build / implement / refactor work that **spans multiple layers** (models, storage, logic, tools, API, UI), or "stacked PRs / stack this". Implement *bottom-up*: one branch & PR per layer, layer N targets layer N−1. Example: `examples/notifications-system.md`. Pairs with **manifest**'s Stack section.

cmux

drive terminals and the browser by command.

WHAT IT DOES

Controls the **cmux** terminal multiplexer from its CLI — manage **workspaces, windows, panes, and surfaces**; send input to any terminal; read terminal screens; and drive the integrated **Playwright-style browser** (navigate, click, fill, snapshot, screenshot). The CLI talks to the running app over a Unix socket; anything cmux does is scriptable.

HOW TO USE

Fires on any mention of **cmux** — “open a workspace”, “split a pane”, “read that terminal”, “send keys to another pane”, or automating a browser tab. Inside a cmux terminal it *autodiscovers* the current workspace/surface/tab, so `cmux send "ls"` targets the current pane; target elsewhere with a short ref like `pane:3`. Start with `cmux tree --all` to inspect the hierarchy.

kagaz

paper, not screen. one surgical accent.

WHAT IT DOES

Kagaz (कागज़) — a personal **editorial-technical** design system: warm paper canvas, a single rust accent under 5% of surface, literary serif display paired with monospace metadata, dot-grid texture, and hairline schematic cards. Reads like a typeset spec sheet, not a SaaS page. Ships a reliable **HTML→PDF converter** with visual QA, plus on-brand SVG charts.

HOW TO USE

Applies to **any** UI, page, dashboard, component, poster, slide deck, report, or PDF — even when not explicitly invoked. Eight invariants: *paper not screen · one accent · warm neutrals · mono as metadata · italic not bold · hairlines not shadows · number everything · one density*. Bundled: `references/` (tokens, components, print, slides), `scripts/` (`html_to_pdf.py`, `charts.py`), brand fonts. **(This guide is rendered in it.)**

dak

a file in, a shareable link out.

WHAT IT DOES

Dak (डाक — "post") turns any local artifact — an HTML report, Markdown notes, a dashboard, a generated PDF, a directory — into a **shareable** `https://` **URL** with one command, hosted free on **Cloudflare Workers**. Two modes: *snapshot* (frozen, immutable, a unique URL per publish) and *live* (a stable URL that updates on redeploy). Prints only the URL to stdout, so it composes cleanly in agent pipelines.

HOW TO USE

Fires whenever output should be **sent, not attached** — "send me a link", "publish this", "host this", or any workflow ending in "…and share the link". `dak.py setup` once, then `dak.py report.html` for a snapshot or `--mode live --slug portfolio` for a stable URL. *Generate → publish → share. Not for* full apps or git-based deploys.

how-to

screenshots or content — into a polished guide.

WHAT IT DOES

Builds a clean, uniform walkthrough **PDF**: a typeset cover, every image on a shadowed white card with a 1px border, optional numbered step headers, UI patch-outs (logos, controls), and an optional slide counter. One consistent look, every page.

HOW TO USE

Two flows, by progressive disclosure. *Images provided* → assemble them (`build_guide_pdf.py`). *No images* → first generate styled slides from a content config (`render_slides.py` , in the kagaz aesthetic), then assemble. Pick the flow; read its reference file. **(This guide was built with it.)**

ian-illustrations

one image, one idea — hand-drawn.

WHAT IT DOES

Ian's **Xiaohei (小黑)** illustration system: reads an article's cognitive anchors and turns them into **16:9 hand-drawn image-gen prompts** — pure white background, black line art, sparse English annotations, one accent flow color, the little deadpan creature performing the core action.

HOW TO USE

Give it the **article content**. It returns a *shot list*, then one prompt per illustration (choosing from 8 composition structures), then a QA self-check. Prompts & annotations are always in English. Example: `examples/prompts.md` — shot list, prompts, and four rendered illustrations.